

C++ Programming Textbook Notes

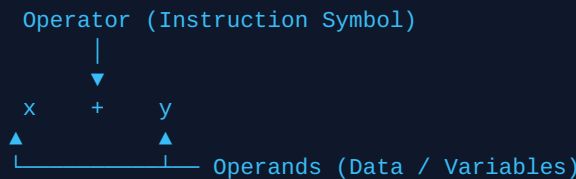
Module: Operators in C++ | Comprehensive Beginner-to-Advanced Guide

1. Overview of Operators in C++

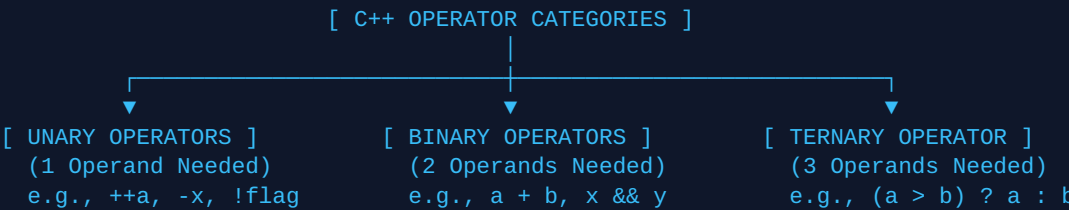
Conceptual Explanation

An **Operator** is a dedicated compiler symbol instructing the Central Processing Unit (CPU) or Arithmetic Logic Unit (ALU) to execute a specific mathematical, logical, relational, or bitwise computation on data inputs.

The data items operated upon are called **Operands**. Operands can be raw literal constants, memory variables, or complex nested expressions.



Classification of Operators by Operand Count



2. Unary Operators (Requires 1 Operand)

Conceptual Explanation

A **Unary Operator** modifies or inspects a single operand. In memory, it reads a single value, evaluates the modification instruction (such as incrementing a byte sequence or negating sign bits), and returns a updated value or memory metadata.

Core Unary Operators Table

Operator	Name	Meaning & Behavior	C++ Code Example
++	Increment	Increases value by 1 (Prefix vs Postfix semantics)	int a = 10; a++; (a becomes 11)
--	Decrement	Decreases value by 1	int a = 10; a--; (a becomes 9)
-	Unary Minus	Negates numerical sign	int b = -a;
!	Logical NOT	Inverts boolean truth value	!(5 > 2) evaluates to false

Operator	Name	Meaning & Behavior	C++ Code Example
&	Address-of	Returns physical RAM address pointer	&a; (e.g., 0x7ffd9000)
*	Indirection (Dereference)	Accesses value stored at target address	*ptr;
sizeof	Size Measurement	Returns exact size in bytes at compile-time	sizeof(int); (Evaluates to 4 bytes)

Prefix vs Postfix Increment Mechanics

Prefix (++x): Increment FIRST → Return NEW value immediately.
Postfix (x++): Copy OLD value → Increment SECOND → Return OLD copy.

Code Example

```
#include <iostream>

int main() {
    int val = 10;

    // Prefix Increment
    int preResult = ++val; // val becomes 11, preResult becomes 11

    // Postfix Increment
    int postResult = val++; // postResult gets 11, val becomes 12

    bool isReady = false;

    std::cout << "Val: " << val << " | Pre: " << preResult << " | Post: " << postResult << "
";
    std::cout << "Logical NOT: " << !isReady << "
"; // Prints 1 (true)
    std::cout << "Byte size: " << sizeof(val) << " bytes
";

    return 0;
}
```

 **Common Gotcha:** Writing multiple increment statements on the same variable in a single expression (e.g., `int x = ++ + ++a;`) causes **undefined behavior** in C++.

3. Binary Operators (Requires 2 Operands)

Conceptual Explanation

A **Binary Operator** operates on two distinct operands placed on either side of the symbol. Binary operators are divided into four core domain groups:

A. Arithmetic Operators

Operator	Operation	Example	Result
+	Addition	$10 + 20$	30
-	Subtraction	$20 - 5$	15
*	Multiplication	$10 * 5$	50
/	Division	$20 / 4$	5 (Truncates decimal for integers)
%	Modulus (Remainder)	$10 \% 3$	1 (Integers only)

B. Relational & Logical Operators (Short-Circuit Logic)

Relational operators (>, <, >=, <=, ==, !=) compare two values and return a bool (true or false). Logical operators (&&, ||) combine boolean conditions using **short-circuit evaluation**:

Short-Circuit Execution:

- Logical AND (A && B) → If A is false, B is NEVER evaluated (Result is false).
- Logical OR (A || B) → If A is true, B is NEVER evaluated (Result is true).

C. Bitwise Operators (Bit-Level Manipulation)

Bit A	Bit B	A & B (AND)	A B (OR)	A ^ B (XOR)	~A (NOT)
0	0	0	0	0	1
0	1	0	1	1	1
1	0	0	1	1	0
1	1	1	1	0	0

Bitwise Shift Mechanics: Shifting `47` Right by 2 Bits (`47 >> 2`)

Decimal 47 in 8-bit binary representation is 00101111. Shifting right by 2 positions drops the rightmost two bits (11) and pads leading zeros on the left:

⚠️ Architectural Rule for Shifts: When right-shifting positive signed numbers, empty slots are padded with zeros. For negative signed numbers, padding behavior depends on compiler/architecture implementation (arithmetic vs logical shift).

Binary Operators Code Example

```
#include <iostream>

int main() {
    int a = 10, b = 3;

    // Arithmetic
    std::cout << "Division: " << (a / b) << "
"; // 3 (integer division)
    std::cout << "Modulus: " << (a % b) << "
"; // 1

    // Bitwise Shift
    int x = 47;
    int shifted = x >> 2;
    std::cout << "47 >> 2 = " << shifted << "
"; // 11

    // Compound Assignment
    a += 5; // Equivalent to a = a + 5; (a becomes 15)
    std::cout << "Updated A: " << a << "
";

    return 0;
}
```

4. Ternary Operator (Requires 3 Operands)

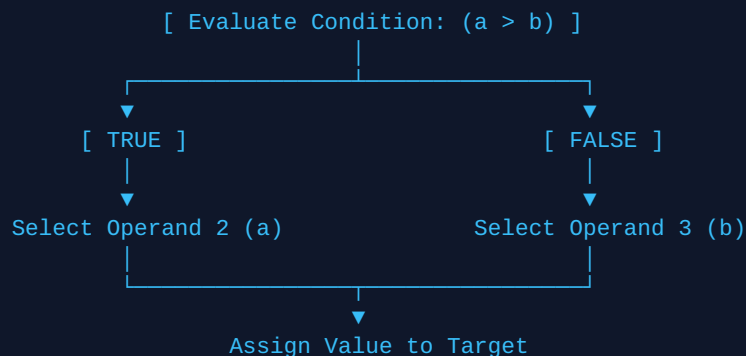
Conceptual Explanation

C++ contains exactly one operator that manages three distinct operands: the **Conditional / Ternary Operator** (`?:`). It provides an inline, expression-based conditional mechanism.

Syntax & Rules

```
(Condition) ? expression_if_true : expression_if_false;
```

Text-Based Logic Diagram



Code Example

```
#include <iostream>

int main() {
    int a = 10, b = 20;

    // Inline maximum selection using ternary operator
    int maxVal = (a > b) ? a : b;

    std::cout << "Maximum value: " << maxVal << "
"; // 20

    return 0;
}
```

 **Best Practice:** Keep ternary expressions simple and readable. Avoid nesting multiple ternary operators inside each other; use standard if-else structures for complex logic branching.

5. Master C++ Operator Reference Table

Functional Category	Operators	Functional Role
Arithmetic	+, -, *, /, %	Executes mathematical computations.
Relational	>, <, >=, <=, ==, !=	Performs value comparison checks.
Logical	&&, , !	Evaluates combined boolean expressions.
Assignment	=, +=, -=, *=, /=, %=	Assigns or updates storage location values.
Increment / Decrement	++, --	Unary shorthand to adjust values by 1.
Conditional	?:	Inline expression branch evaluation.
Bitwise	&, , ^, ~, <<, >>	Performs direct binary digit manipulation.
Special / Memory	sizeof(), &, *, ->, .	Memory sizing, pointer dereferencing, and member selection.